

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ»

КАФЕДРА ТЕОРЕТИЧЕСКОЙ И ПРИКЛАДНОЙ ИНФОРМАТИКИ

Лабораторная работа № 4
по дисциплине «Теория информации и криптография»
на тему «Генерация криптографически безопасной
псевдослучайной последовательности»

Факультет: ФПМИ

Группа: ПМИ-32

Студенты: Весёлый Д., Ворончук И., Лыкова М.

Бригада №2

Преподаватели: Авдеев Т. В., Кутузова И. А.

НОВОСИБИРСК 2025

Цель работы: Изучить алгоритмы и получить навыки генерации криптографически безопасной псевдослучайной последовательности. Научиться тестировать полученную последовательность на равномерность и случайность.

Задание:

- I. Реализовать приложение с графическим интерфейсом, позволяющее выполнять следующие действия.
 1. Генерировать псевдослучайную последовательность с помощью заданного в варианте алгоритма:
 - 1) все входные параметры генератора должны задаваться из файла или вводиться в приложении;
 - 2) сгенерированная последовательность, состоящая из 0 и 1, должна сохраняться в файл.
 2. Проверять полученную псевдослучайную последовательность на равномерность и случайность с помощью трёх рассмотренных тестов:
 - 1) результат проверки каждого теста должен отображаться в приложении;
 - 2) все вычисляемые промежуточные значения (все шаги алгоритма теста) могут отображаться в приложении или сохраняться в файл.
- II. С помощью реализованного приложения выполнить следующие задания.
 1. Протестировать правильность работы разработанного приложения.
 2. Сгенерировать последовательность из не менее 10 000 бит и исследовать её на равномерность и случайность.
 3. Сделать вывод о случайности сгенерированной последовательности и о возможности её использования в качестве криптографически безопасной псевдослучайной последовательности.

Вариант	Алгоритм
2	ANSI X9.17

Описание метода решения заданий.

Для выполнения работы используется алгоритм ANSI X9.17 для генерации двоичной псевдослучайной последовательности. Данный алгоритм использует в качестве односторонней функции алгоритм TripleDES (3DES), стойкость которого основана на многократном применении блочного шифрования по схеме “шифрование - дешифрование - шифрование” (E-D-E).

Алгоритм обеспечивает непредсказуемость следующего бита даже при знании всех предыдущих значений и структуры алгоритма. Он включает

временную метку (дату), что делает последовательность зависимой от момента генерации и усложняет её воспроизведение.

Входные параметры:

S_0 - секретное 64-битное начальное значение (seed);

$K = (K_1, K_2)$ - составной 128-битный ключ для 3DES;

m - количество генерируемых 64-битных слов;

d - 64-битное представление даты и времени;

Функция шифрования:

Односторонняя функция F_K реализуется через 3DES следующим образом:

$F_K(M) = E_{K1}(D_{K2}(E_{K1}(M)))$, где E - операция шифрования DES, D - операция дешифрования DES.

Алгоритм генерации:

1. Инициализация: Вычисляется вспомогательное значение Temp путем шифрования даты: $Temp = F_K(d)$.
2. Рекуррентный процесс (для $i = 1, \dots, m$):
 - a. Генерация выходного слова x_i : $x_i = F_K(Temp \text{ XOR } S_{i-1})$
 - b. Обновление внутреннего состояния S_i для следующего шага: $S_i = F_K(x_i \text{ XOR } Temp)$

Выходные данные: последовательность $x_1, x_2, \dots, x_m$, представленная в виде битовой строки длиной $64*m$ бит.

Статистические тесты NIST

Для оценки качества сгенерированной последовательности применяются три теста из набора NIST.

Предварительная обработка:

Перед тестированием битовая строка $\varepsilon = \{\varepsilon_i\}$, где ε преобразуется в последовательность $X = \{X_i\}$, где $X_i \in \{-1, 1\}$ по правилу: $X_i = 2*\varepsilon_i - 1$.

1. Частотный тест.

Цель: Оценка равномерности распределения нулей и единиц в последовательности.

Статистика: $S = |S_n| / \sqrt{n}$, где S_n - сумма элементов преобразованной последовательности.

Критерий прохождения:

$S \leq 1.82138636$ - пройден.

$S > 1.82138636$ - провален (последовательность является неслучайной).

2. Тест на последовательность одинаковых бит

Цель: Проверка частоты чередования цепочек (серий) одинаковых битов.

Оценивает, не чередуются ли биты слишком часто.

Статистика: $S = |V_n - 2^n \pi(1-\pi)| / (2 \sqrt{2^n \pi(1-\pi)})$, где

π - доля единиц в исходной последовательности, π = кол-во единиц/ n . V_n - общее количество серий (цепочек одинаковых битов) в последовательности.

Критерий прохождения:

$S \leq 1.82138636$ - пройден.

$S > 1.82138636$ - провален (последовательность является неслучайной).

3. Расширенный тест на произвольные отклонения

Цель: Проверить, не отклоняется ли кумулятивная сумма битов слишком сильно от нуля. Оценивается частота посещения состояний на числовой оси.

Строится кумулятивная сумма S'_k . Подсчитывается количество посещений каждого состояния j (от -9 до +9, исключая 0).

Статистика: $Y_j = |\xi_j - L| / \sqrt{2L(4|j|-2)}$, где L - расчетное ожидаемое количество посещений (зависит от количества нулей в кумулятивной сумме).

Тест считается пройденным, если все 18 значений $Y_j \leq 1.82138636$.

Описание разработанного программного средства.

Программное средство предназначено для генерации криптографически стойких псевдослучайных последовательностей на базе алгоритма ANSI X9.17 с использованием блочного шифра TripleDES (3DES), а также для статистической верификации качества выходных данных методами, адаптированными из набора тестов NIST.

Алгоритмическая реализация

Базовый шифр DES

Полностью воспроизводит спецификацию FIPS 46-3: Начальная/конечная перестановки (IP, FP), расширение E ($32 \rightarrow 48$ бит), 8 S-блоков ($6 \rightarrow 4$ бита), перестановка P (32 бита), расписание ключей (PC1, PC2,

циклические сдвиги SHIFTS). Корректная обработка битовой нумерации (1-индексация стандарта DES преобразована в 0-индексацию Python).

```
def des_encrypt(block: int, key: int) -> int:
    """DES шифрование одного 64-битного блока."""
    # Генерация подключей
    K = [0] * 16
    pc1 = _permute(key, PC1, 56)
    C = pc1 >> 28
    D = pc1 & 0xFFFFFFFF

    for i in range(16):
        shift = SHIFTS[i]
        C = ((C << shift) | (C >> (28 - shift))) & 0xFFFFFFFF
        D = ((D << shift) | (D >> (28 - shift))) & 0xFFFFFFFF
        K[i] = _permute((C << 28) | D, PC2, 48)

    # Начальная перестановка и разделение на L и R
    block = _permute(block, IP, 64)
    L = block >> 32
    R = block & 0xFFFFFFFF

    # 16 раундов шифрования
    for i in range(16):
        # Expansion + XOR с ключом
        er = _permute(R, E, 48) ^ K[i]
        # Применение S-блоков
        s_out = 0
        for j in range(8):
            row = (((er >> (42 - 6 * j + 5)) & 1) << 1) | ((er >>
(42 - 6 * j)) & 1)
            col = (er >> (42 - 6 * j + 1)) & 0xF
            s_out = (s_out << 4) | S[j][row * 16 + col]
        # Перестановка P + XOR с L
        next_R = L ^ _permute(s_out, P, 32)
        L = R
        R = next_R

    # Финальная перестановка
    return _permute((R << 32) | L, FP, 64)
```

TripleDES

Используется схема Encrypt-Decrypt-Encrypt с двумя независимыми ключами: $F_K(M) = E_K1(D_K2(E_K1(M)))$.

Длина составного ключа: 128 бит ($K1 \parallel K2$).

```
def encrypt_3des(data: int, k1: int, k2: int) -> int:
    """3DES: E_K1( D_K2( E_K1(M) ) )"""
    return des_encrypt(des_decrypt(des_encrypt(data, k1), k2), k1)
```

Генератор ANSI X9.17

Реализует рекуррентные соотношения стандарта:

1. Инициализация: $Temp = F_K(d)$, где d — текущее время в миллисекундах.
2. Цикл генерации ($i = 1 \dots m$):
 - a. $x_i = F_K(Temp \oplus S_{i-1})$
 - b. $S_i = F_K(x_i \oplus Temp)$
3. Вывод: конкатенация 64-битных слов x_i в битовую строку.

Модуль статистического тестирования

Три теста проверяют свойства случайности. Пороговое значение для всех статистик: 1.82138636 (уровень значимости $\alpha = 0.01$).

Особенности реализации:

- Предварительное преобразование $\varepsilon \rightarrow X$: $0 \rightarrow -1$, $1 \rightarrow +1$
- В расширенном тесте анализируются 18 состояний $j \in \{-9..-1, 1..9\}$
- Результаты выводятся в табличном виде с пометкой прошёл/не прошёл для каждого состояния.

Описание проведённых исследований.

Тест 1: Проверка базовой генерации

Входные данные:

- $K1 = 1234567890ABCDEF$ (hex)
- $K2 = FEDCBA0987654321$ (hex)
- $s_0 = 0011223344556677$ (hex)
- $m = 1$ (одно 64-битное слово)

Ожидаемый результат:

- Длина выходной строки: 64 символа;
- Строка содержит только символы '0' и '1'.

d (текущая дата/время (мс)): 2026-04-07 18:20:43.767 =
0x0000019D67AC61F8 или 00000000 00000000 00000001 10011101
01100111 10101100 01100001 11111000

Первый ключ K_1 : $0x1234567890ABCDEF_{16} = 00010010\ 00110100\ 01010110\ 01111000\ 10010000\ 10101011\ 11001101\ 11101111_2$

Второй ключ K_2 : $0xFEDCBA0987654321_{16} = 11111110\ 11011100\ 10111010\ 00001001\ 10000111\ 01100101\ 01000011\ 00100001_2$

Начальное состояние S_0 : $0x0011223344556677_{16} = 00000000\ 00010001\ 00100010\ 00110011\ 01000100\ 01010101\ 01100110\ 01110111_2$

1. Вычисление temp.

Сначала необходимо зашифровать дату, формула: $Temp = 3DES(d, K1, K2)$, то есть зашифровать $K1 \rightarrow$ Расшифровать $K2 \rightarrow$ Зашифровать $K1$.

На вход поступает дата d .

Результат (в задании было сказано можно использовать готовую реализацию): 0x0000026E9B5C92F4.

2. Генерация выходного числа x_1

Сгенерируем само случайное число, формула: $x_1 = 3DES(Temp \text{ XOR } S_0)$.

$Temp$: 0x0000026E9B5C92F4

S_0 : 0x0011223344556677

Их XOR: 0x0011205DDF09F483

Шифруем полученный результат через 3DES, получаем $x_1 = 0x002210AEEF06F843$.

3. Переводим из шестнадцатеричной системы счисления в двоичную:
 $0x002210AEEF06F843_{16} =$
 $0000000000100010000100001010111011101111000001101111100001000011_2$

4. Обновляем состояние s_1
Обновляем внутреннее состояние генератора через формулу: $S_1 = 3DES(x_1 \text{ XOR Temp})$.

x_1 : $0x002210AEEF06F843$

Temp: $0x0000026E9B5C92F4$

Их XOR: $0x002212C0745A6AB7$

Шифруем результат XOR через 3DES: $s_1 = 0x001121C0B8A5957B$

Вывод программы:

Генерация ПСП (ANSI X9.17)

Статистические тесты NIST

Загрузить параметры из файла...

Ключ K1 (16 hex-символов):

1234567890ABCDEF

Ключ K2 (16 hex-символов):

FEDCBA0987654321

Начальное значение s0 (16 hex-символов):

0011223344556677

Количество 64-битных блоков m:

—

1

+

Сгенерировать последовательность

Сгенерированная битовая последовательность...

0000000000100010000100001010111011101111000001101111100001000011

Сохранить последовательность...

$0000000000100010000100001010111011101111000001101111100001000011$

Ручное решение совпадает с выводом программы.

Статистическое тестирование (набор NIST).

Начальные значения:

$$K_1 = A1B2C3D4E5F60789_{16}$$

$$K_2 = 9876543210FEDCBA_{16}$$

$$S_0 = 1122334455667788$$

$$m = 157 \text{ (} 157 * 64 \text{ бита} = 10048 \text{ бит)}$$

Получившаяся последовательность:

0101111101000...011111100101101₂

Получившаяся длина:

```
000010001001100100001010.  
[>>> len(a)  
10048
```

Тест 1: Частотный тест (Monobit Frequency Test)

Преобразованная последовательность X:

-11-111111-11-1-1-1...-1111111-1-11-111-11

Сумма элементов преобразованной последовательности: $S_n = 110$

Статистика: $S = |S_n|/\sqrt{n} = 110/\sqrt{10048} = 1.097$

$S = 1.097 \leq 1.827 \dots$ - тест пройден.

Тест 2: Тест на последовательность одинаковых бит (Runs Test)

Вычислим долю единиц в последовательности:

$$\pi = 1/n * \sum(\epsilon_j) = (1/10048) * 5079 = 0.5$$

Количество серий: $V_n = 1 + \sum r(k)$, где $r(k)=1$ если $\epsilon_k \neq \epsilon_{k+1}$, $V_n = 4994$

Ожидаемое количество серий: $E = 2 * n * \pi * (1-\pi) / 1 = 2 * 10048 * 0.505474 * 0.494526 \approx 5019$.

Статистика: $S = |V_n - 2 * n * \pi * (1-\pi)| / \sqrt{2 * n * \pi * (1-\pi)} = 0.414804$

$S = 0.414804 \leq 1.82138636$ - тест пройден.

Тест 3: Расширенный тест на произвольные отклонения.

Преобразованная последовательность X:

-11-111111-11-1-1-1...-1111111-1-11-111-11

Кумулятивные суммы:

$$S_1 = -1, S_2 = -1 + 1 = 0, S_3 = 0 - 1 = -1, S_4 = -1 + 1 = 0, S_5 = 0 + 1 = 1.$$

Формируем $S' = [0, S_1, \dots, S_n, 0] = [0, -1, 0, \dots, 1, 0]$.

Количество нулей в S' : 56, $L = 56 - 1 = 55$.

Для каждого состояния от -9 до +9 (кроме 0) считаем, сколько раз встретилось число:

$S_5 = 0 + 1 = 1$	Посещений (ξ_j)	Ожидаемо (L)	Отклонение
-9	26	55	-20
-8	35	55	-14
...	...	...	...
-1	54	55	-1
+1	56	55	+1
...	...	...	...
+9	37	55	-18

Вычисление статистики Y_j :

Для $j = -9$: $Y_{-9} = 0.4742$

...

Для $j = -1$: $Y_{-1} = 0.06742$

...

Для $j = +1$: $Y_{+1} = 0.0674$

...

Для $j = +9$: $Y_{+9} = 0.2943$

Все 18 значений ≤ 1.82138636 , все тесты пройдены.

Сгенерированная последовательность выдаёт сбалансированную последовательность, обеспечивает естественное чередование битов и не имеет аномалий в распределении состояний. Последовательность статистически неотличима от случайной и пригодна для криптографических целей.

Вывод программы:

Лог тестов

===== РЕЗУЛЬТАТЫ ТЕСТОВ NIST =====

[1] ЧАСТОТНЫЙ ТЕСТ

Длина $n = 10048$

Сумма $S_n = 110$

Статистика $S = 1.097369$

РЕЗУЛЬТАТ: ПРОЙДЕН ($S \leq 1.82138636$)

[2] ТЕСТ НА ПОСЛЕДОВАТЕЛЬНОСТЬ ОДИНАКОВЫХ БИТ

Частота единиц $p_i = 0.505474$

Количество цепочек $V_n = 4994$

Статистика $S = 0.414804$

РЕЗУЛЬТАТ: ПРОЙДЕН ($S \leq 1.82138636$)

[3] РАСШИРЕННЫЙ ТЕСТ НА ПРОИЗВОЛЬНЫЕ ОТКЛОНЕНИЯ

Количество нулей в $S' (L) = 55$

Состояние(j)	Встреч(xi)	Статистика(Y_j)	Результат
--------------	------------	-----------------	-----------

-9	26	0.474201	PASS
-8	35	0.348155	PASS
-7	41	0.261785	PASS
-6	43	0.243935	PASS
-5	45	0.224733	PASS
-4	54	0.025482	PASS
-3	55	0.000000	PASS
-2	53	0.077850	PASS
-1	54	0.067420	PASS
1	56	0.067420	PASS
2	54	0.038925	PASS
3	65	0.301511	PASS
4	67	0.305788	PASS
5	59	0.089893	PASS
6	50	0.101639	PASS
7	37	0.336581	PASS
8	35	0.348155	PASS
9	37	0.294331	PASS

ИТОГОВЫЙ РЕЗУЛЬТАТ: ПРОЙДЕН ($\text{Все } Y_j \leq 1.82138636$)

Тестирование программы.

Тест 1 (одно слово)

- K1: 0123456789ABCDEF
- K2: FEDCBA9876543210
- s0: 1234567890ABCDEF
- m: 1

Ожидаемый результат:

- Длина выходной строки: 64 бита.
- Строка содержит только символы '0' и '1'.

Результат программы:

1000100001110111111000110010101010111100111011001100001001010100

Длина строки = 64.

Загрузить параметры из файла...

Ключ K1 (16 hex-символов): 0123456789ABCDEF

Ключ K2 (16 hex-символов): FEDCBA9876543210

Начальное значение s0 (16 hex-символов): 1234567890ABCDEF

Количество 64-битных блоков m: 1

Готово: 64 бит

Сгенерировать последовательность

Результат

1000100001110111111000110010101010111100111011001100001001010100

Тест 2: ≥ 10000 бит

- K1: A1B2C3D4E5F60718
- K2: 81706F5E4D3C2B1A
- s0: 1122334455667788
- m: 160 ($160 * 64 = 10\,240$ бит)

Ожидаемый результат:

- Длина выходной строки: 10240 бит.
- Строка содержит только символы '0' и '1'.

Результат программы:

00100010101010001011011111001110100011000101110100010100100000001
110110011001001011100110111001111011110000010001111110110110000110
01011101110001001000011111100010001010001000100011111101010000101
0110...

Длина равна 10240 бит.

Генерация ПСП (ANSI X9.17)

Статистические тесты NIST

Загрузить параметры из файла...

Ключ K1 (16 hex-символов):

A1B2C3D4E5F60718

Ключ K2 (16 hex-символов):

81706F5E4D3C2B1A

Начальное значение s0 (16 hex-символов):

1122334455667788

Количество 64-битных блоков m:

160

Готово: 10240 бит

Сгенерировать последовательность

Результат

001101110011110111100000100011111011011000011001011101110001001000011111100010001010001000100011111101010000101011011010

Тест 3: $m = 0$ или $m = -5$:

Ожидаемый результат: программа выводит сообщение об ошибке.

Результат программы: сообщение об ошибке (и невозможность ввести такие числа).

Тест 4: некорректный hex-ключ:

- 0123456789ABCD--

Ожидаемый результат: программа выводит сообщение об ошибке.

Результат программы: сообщение об ошибке.

Вывод

В ходе данной лабораторной работы были изучены алгоритмы генерации криптографически безопасных псевдослучайных последовательностей и реализован алгоритм ANSI X9.17. Была сгенерирована последовательность длиной не менее 10 000 бит с использованием секретных ключей и временной метки для обеспечения уникальности. Полученная последовательность была проверена на равномерность и случайность с помощью трёх статистических тестов из набора NIST. Все тесты были успешно пройдены, что подтверждает статистическую неотличимость последовательности от истинно случайной. Благодаря непредсказуемости следующего бита и использованию шифра 3DES, последовательность может считаться криптографически стойкой. Таким образом, сгенерированная последовательность пригодна для использования в криптографических приложениях, таких как генерация сеансовых ключей или векторов инициализации.

Код программы.

generator_ansi.py

```
import math
import time
from datetime import datetime

# ТАБЛИЦЫ DES
IP = [
    58, 50, 42, 34, 26, 18, 10, 2, 60, 52, 44, 36, 28,
    20, 12, 4, 62, 54, 46, 38, 30, 22, 14, 6, 64, 56,
    48, 40, 32, 24, 16, 8, 57, 49, 41, 33, 25, 17, 9,
    1, 59, 51, 43, 35, 27, 19, 11, 3, 61, 53, 45, 37,
    29, 21, 13, 5, 63, 55, 47, 39, 31, 23, 15, 7,
]

FP = [
    40, 8, 48, 16, 56, 24, 64, 32, 39, 7, 47, 15, 55,
    23, 63, 31, 38, 6, 46, 14, 54, 22, 62, 30, 37, 5,
    45, 13, 53, 21, 61, 29, 36, 4, 44, 12, 52, 20, 60,
    28, 35, 3, 43, 11, 51, 19, 59, 27, 34, 2, 42, 10,
    50, 18, 58, 26, 33, 1, 41, 9, 49, 17, 57, 25,
]

E = [
    32, 1, 2, 3, 4, 5, 4, 5, 6, 7, 8, 9,
    8, 9, 10, 11, 12, 13, 12, 13, 14, 15, 16, 17,
    16, 17, 18, 19, 20, 21, 20, 21, 22, 23, 24, 25,
    24, 25, 26, 27, 28, 29, 28, 29, 30, 31, 32, 1,
]

P = [
    16, 7, 20, 21, 29, 12, 28, 17, 1, 15, 23,
    26, 5, 18, 31, 10, 2, 8, 24, 14, 32, 27,
    3, 9, 19, 13, 30, 6, 22, 11, 4, 25,
]

PC1 = [
    57, 49, 41, 33, 25, 17, 9, 1, 58, 50, 42, 34, 26, 18, 10, 2,
    59, 51, 43, 35, 27, 19, 11, 3, 60, 52, 44, 36, 63, 55, 47, 39,
    31, 23, 15, 7, 62, 54, 46, 38, 30, 22, 14, 6, 61, 53, 45, 37,
    29, 21, 13, 5, 28, 20, 12, 4,
]

PC2 = [
    14, 17, 11, 24, 1, 5, 3, 28, 15, 6, 21, 10,
```

```

23, 19, 12, 4, 26, 8, 16, 7, 27, 20, 13, 2,
41, 52, 31, 37, 47, 55, 30, 40, 51, 45, 33, 48,
44, 49, 39, 56, 34, 53, 46, 42, 50, 36, 29, 32,
]

```

```

SHIFTS = [1, 1, 2, 2, 2, 2, 2, 2, 1, 2, 2, 2, 2, 2, 1]

```

```

S = [
[
14, 4, 13, 1, 2, 15, 11, 8, 3, 10, 6, 12, 5, 9, 0, 7,
0, 15, 7, 4, 14, 2, 13, 1, 10, 6, 12, 11, 9, 5, 3, 8,
4, 1, 14, 8, 13, 6, 2, 11, 15, 12, 9, 7, 3, 10, 5, 0,
15, 12, 8, 2, 4, 9, 1, 7, 5, 11, 3, 14, 10, 0, 6, 13,
],
[
15, 1, 8, 14, 6, 11, 3, 4, 9, 7, 2, 13, 12, 0, 5, 10,
3, 13, 4, 7, 15, 2, 8, 14, 12, 0, 1, 10, 6, 9, 11, 5,
0, 14, 7, 11, 10, 4, 13, 1, 5, 8, 12, 6, 9, 3, 2, 15,
13, 8, 10, 1, 3, 15, 4, 2, 11, 6, 7, 12, 0, 5, 14, 9,
],
[
10, 0, 9, 14, 6, 3, 15, 5, 1, 13, 12, 7, 11, 4, 2, 8,
13, 7, 0, 9, 3, 4, 6, 10, 2, 8, 5, 14, 12, 11, 15, 1,
13, 6, 4, 9, 8, 15, 3, 0, 11, 1, 2, 12, 5, 10, 14, 7,
1, 10, 13, 0, 6, 9, 8, 7, 4, 15, 14, 3, 11, 5, 2, 12,
],
[
7, 13, 14, 3, 0, 6, 9, 10, 1, 2, 8, 5, 11, 12, 4, 15,
13, 8, 11, 5, 6, 15, 0, 3, 4, 7, 2, 12, 1, 10, 14, 9,
10, 6, 9, 0, 12, 11, 7, 13, 15, 1, 3, 14, 5, 2, 8, 4,
3, 15, 0, 6, 10, 1, 13, 8, 9, 4, 5, 11, 12, 7, 2, 14,
],
[
2, 12, 4, 1, 7, 10, 11, 6, 8, 5, 3, 15, 13, 0, 14, 9,
14, 11, 2, 12, 4, 7, 13, 1, 5, 0, 15, 10, 3, 9, 8, 6,
4, 2, 1, 11, 10, 13, 7, 8, 15, 9, 12, 5, 6, 3, 0, 14,
11, 8, 12, 7, 1, 14, 2, 13, 6, 15, 0, 9, 10, 4, 5, 3,
],
[
12, 1, 10, 15, 9, 2, 6, 8, 0, 13, 3, 4, 14, 7, 5, 11,
10, 15, 4, 2, 7, 12, 9, 5, 6, 1, 13, 14, 0, 11, 3, 8,
9, 14, 15, 5, 2, 8, 12, 3, 7, 0, 4, 10, 1, 13, 11, 6,
4, 3, 2, 12, 9, 5, 15, 10, 11, 14, 1, 7, 6, 0, 8, 13,
],
[
4, 11, 2, 14, 15, 0, 8, 13, 3, 12, 9, 7, 5, 10, 6, 1,

```

```

        13, 0, 11, 7, 4, 9, 1, 10, 14, 3, 5, 12, 2, 15, 8, 6,
        1, 4, 11, 13, 12, 3, 7, 14, 10, 15, 6, 8, 0, 5, 9, 2,
        6, 11, 13, 8, 1, 4, 10, 7, 9, 5, 0, 15, 14, 2, 3, 12,
    ],
    [
        13, 2, 8, 4, 6, 15, 11, 1, 10, 9, 3, 14, 5, 0, 12, 7,
        1, 15, 13, 8, 10, 3, 7, 4, 12, 5, 6, 11, 0, 14, 9, 2,
        7, 11, 4, 1, 9, 12, 14, 2, 0, 6, 10, 13, 15, 3, 5, 8,
        2, 1, 14, 7, 4, 10, 8, 13, 15, 12, 9, 0, 3, 5, 6, 11,
    ],
]

```

Вспомогательные функции DES

```

def _permute(input_val: int, table: list[int], size: int) -> int:
    """Перестановка битов по заданной таблице.

    size -- размер ВЫХОДА (количество бит в результате).
    Таблицы DES используют 1-индексацию битов, где бит 1 = старший бит
    входа.
    Размер входа определяется максимальным индексом в таблице.
    """
    res = 0
    # Определяем размер входа по максимальному индексу в таблице
    input_size = max(table)
    for i in range(size):
        bit_index = input_size - table[i] # table[i] = 1 => старший бит
        res = (res << 1) | ((input_val >> bit_index) & 1)
    return res

```

```

def des_encrypt(block: int, key: int) -> int:
    """DES шифрование одного 64-битного блока."""
    # Генерация подключей
    K = [0] * 16
    pc1 = _permute(key, PC1, 56)
    C = pc1 >> 28
    D = pc1 & 0xFFFFFFFF

    for i in range(16):
        shift = SHIFTS[i]
        C = ((C << shift) | (C >> (28 - shift))) & 0xFFFFFFFF
        D = ((D << shift) | (D >> (28 - shift))) & 0xFFFFFFFF
        K[i] = _permute((C << 28) | D, PC2, 48)

    # Начальная перестановка

```



```

block = _permute(block, IP, 64)
L = block >> 32
R = block & 0xFFFFFFFF

# 16 раундов
for i in range(16):
    # Expansion + XOR с ключом
    er = _permute(R, E, 48) ^ K[i]
    # S-блоки
    s_out = 0
    for j in range(8):
        row = (((er >> (42 - 6 * j + 5)) & 1) << 1) | ((er >> (42 - 6
* j)) & 1)
        col = (er >> (42 - 6 * j + 1)) & 0xF
        s_out = (s_out << 4) | S[j][row * 16 + col]
    # Перестановка P + XOR с L
    next_R = L ^ _permute(s_out, P, 32)
    L = R
    R = next_R

# Финальная перестановка (R << 32 | L, не L << 32 | R)
return _permute((R << 32) | L, FP, 64)

```

```

def des_decrypt(block: int, key: int) -> int:
    """DES дешифрование одного 64-битного блока."""
    # Генерация подключей (такая же, как в encrypt)
    K = [0] * 16
    pc1 = _permute(key, PC1, 56)
    C = pc1 >> 28
    D = pc1 & 0xFFFFFFFF

    for i in range(16):
        shift = SHIFTS[i]
        C = ((C << shift) | (C >> (28 - shift))) & 0xFFFFFFFF
        D = ((D << shift) | (D >> (28 - shift))) & 0xFFFFFFFF
        K[i] = _permute((C << 28) | D, PC2, 48)

    # Начальная перестановка
    block = _permute(block, IP, 64)
    L = block >> 32
    R = block & 0xFFFFFFFF

    # 16 раундов в обратном порядке
    for i in range(15, -1, -1):
        er = _permute(R, E, 48) ^ K[i]

```

```

        s_out = 0
        for j in range(8):
            row = (((er >> (42 - 6 * j + 5)) & 1) << 1) | ((er >> (42 - 6
* j)) & 1)
            col = (er >> (42 - 6 * j + 1)) & 0xF
            s_out = (s_out << 4) | S[j][row * 16 + col]
            next_R = L ^ _permute(s_out, P, 32)
            L = R
            R = next_R

        return _permute((R << 32) | L, FP, 64)

```

```

def encrypt_3des(data: int, k1: int, k2: int) -> int:
    """3DES: E_K1( D_K2( E_K1(M) ) )"""
    return des_encrypt(des_decrypt(des_encrypt(data, k1), k2), k1)

```

Генерация ANSI X9.17

```

def generate_ansi(k1_hex: str, k2_hex: str, s0_hex: str, m: int,
                  progress_cb=None) -> str:
    """
    Генерация псевдослучайной последовательности по алгоритму ANSI X9.17.
    Возвращает битовую строку длиной m*64.
    progress_cb -- опциональный callback(current, total) для отображения
    прогресса.
    """
    k1 = int(k1_hex, 16)
    k2 = int(k2_hex, 16)
    s_prev = int(s0_hex, 16)

    # Текущее время в миллисекундах (аналог
    QDateTime::currentMSecsSinceEpoch)
    d = int(time.time() * 1000)
    temp = encrypt_3des(d, k1, k2)

    result = []
    for i in range(m):
        x_i = encrypt_3des(temp ^ s_prev, k1, k2)
        s_prev = encrypt_3des(x_i ^ temp, k1, k2)

    # 64 бита, старший первым
    for b in range(63, -1, -1):
        result.append('1' if (x_i >> b) & 1 else '0')

```

```

        if progress_cb:
            progress_cb(i + 1, m)

```

```

    return ''.join(result)

```

Статистические тесты NIST

```

def frequency_test(seq: str) -> str:
    """[1] Частотный (монобитный) тест."""
    log = "[1] Частотный тест\n"
    n = len(seq)
    Sn = sum(1 if c == '1' else -1 for c in seq)

    S = abs(Sn) / math.sqrt(n)

    log += f"Длина n = {n}\n"
    log += f"Сумма Sn = {Sn}\n"
    log += f"Статистика S = {S:.6f}\n"

    if S <= 1.82138636:
        log += "Результат: пройден (S <= 1.82138636)\n"
    else:
        log += "Результат: провален (S > 1.82138636)\n"

    return log

```

```

def runs_test(seq: str) -> str:
    """[2] Тест на последовательность одинаковых бит (runs test)."""
    log = "[2]Тест на последовательность одинаковых бит\n"
    n = len(seq)
    ones = seq.count('1')

    pi = ones / n

    Vn = 1
    for i in range(n - 1):
        if seq[i] != seq[i + 1]:
            Vn += 1

    S = abs(Vn - 2.0 * n * pi * (1.0 - pi)) / \
        (2.0 * math.sqrt(2.0 * n) * pi * (1.0 - pi))

    log += f"Частота единиц pi = {pi:.6f}\n"
    log += f"Количество цепочек Vn = {Vn}\n"
    log += f"Статистика S = {S:.6f}\n"

```

```

if S <= 1.82138636:
    log += "Результат: пройден (S <= 1.82138636)\n"
else:
    log += "Результат: провален (S > 1.82138636)\n"

return log

def extended_deviation_test(seq: str) -> str:
    """[3] Расширенный тест на произвольные отклонения."""
    log = "[3] РАСШИРЕННЫЙ ТЕСТ НА ПРОИЗВОЛЬНЫЕ ОТКЛОНЕНИЯ\n"

    s_prime = [0]
    current_sum = 0
    for c in seq:
        current_sum += 1 if c == '1' else -1
        s_prime.append(current_sum)
    s_prime.append(0)

    L = -1
    xi: dict[int, int] = {}
    for val in s_prime:
        if val == 0:
            L += 1
        if -9 <= val <= 9 and val != 0:
            xi[val] = xi.get(val, 0) + 1

    log += f"Количество нулей в S' (L) = {L}\n\n"
    log += "Состояние(j) | Встреч(xi) | Статистика(Y_j) | Результат\n"
    log += "-----\n"

    all_passed = True
    for j in range(-9, 10):
        if j == 0:
            continue

        x_j = xi.get(j, 0)
        Y_j = abs(x_j - L) / math.sqrt(2.0 * L * (4.0 * abs(j) - 2.0))

        res_str = "PASS" if Y_j <= 1.82138636 else "FAIL"
        if Y_j > 1.82138636:
            all_passed = False

    log += f"{j:>12} | {x_j:>10} | {Y_j:>15.6f} | {res_str}\n"

```

```

log += "\nИтог: "
log += "Пройден (Все Y_j <= 1.82138636)\n" if all_passed \
    else "Провален (Есть Y_j > 1.82138636)\n"

return log


def run_tests(bit_sequence: str) -> str:
    """Запуск всех трёх тестов."""
    if not bit_sequence:
        return "Ошибка: Пустая последовательность."

    log = "Результаты тестов NIST\n\n"
    log += frequency_test(bit_sequence) + "\n"
    log += runs_test(bit_sequence) + "\n"
    log += extended_deviation_test(bit_sequence)

    return log


# Файловые операции

def read_file_content(file_path: str) -> str:
    """Чтение текста из файла."""
    try:
        with open(file_path, 'r', encoding='utf-8') as f:
            return f.read()
    except (IOError, OSError):
        return ""


def save_file_content(file_path: str, content: str) -> bool:
    """Запись текста в файл."""
    try:
        with open(file_path, 'w', encoding='utf-8') as f:
            f.write(content)
        return True
    except (IOError, OSError):
        return False


# CLI-интерфейс

def main():
    import sys
    import os

```

```

# Путь к config.txt относительно расположения скрипта
script_dir = os.path.dirname(os.path.abspath(__file__))
config_path = os.path.join(script_dir, "..", "config.txt")

if len(sys.argv) > 1:
    config_path = sys.argv[1]

if len(sys.argv) >= 5:
    # Параметры из командной строки
    k1_hex = sys.argv[1]
    k2_hex = sys.argv[2]
    s0_hex = sys.argv[3]
    m = int(sys.argv[4])
else:
    # Попытка прочитать из config
    content = read_file_content(config_path)
    if content:
        params = content.strip().split()
        if len(params) >= 4:
            k1_hex = params[0]
            k2_hex = params[1]
            s0_hex = params[2]
            m = int(params[3])
        else:
            print("Ошибка: config.txt должен содержать 4 параметра:
K1 K2 s0 m")
            sys.exit(1)
        else:
            print(f"Не найден файл {config_path}")
            sys.exit(1)
    print(f"K1 = {k1_hex}")
    print(f"K2 = {k2_hex}")
    print(f"s0 = {s0_hex}")
    print(f"m = {m}")
    print()
    # Генерация
    print("Генерация последовательности...")
    seq = generate_ansi(k1_hex, k2_hex, s0_hex, m)
    print(f"Длина последовательности: {len(seq)} бит")
    print()
    # Тесты
    print(run_tests(seq))

if __name__ == "__main__":

```

```
main()
```